ITS Documentation

Intelligent Trading Strategies system

Generated: 2026-05-22

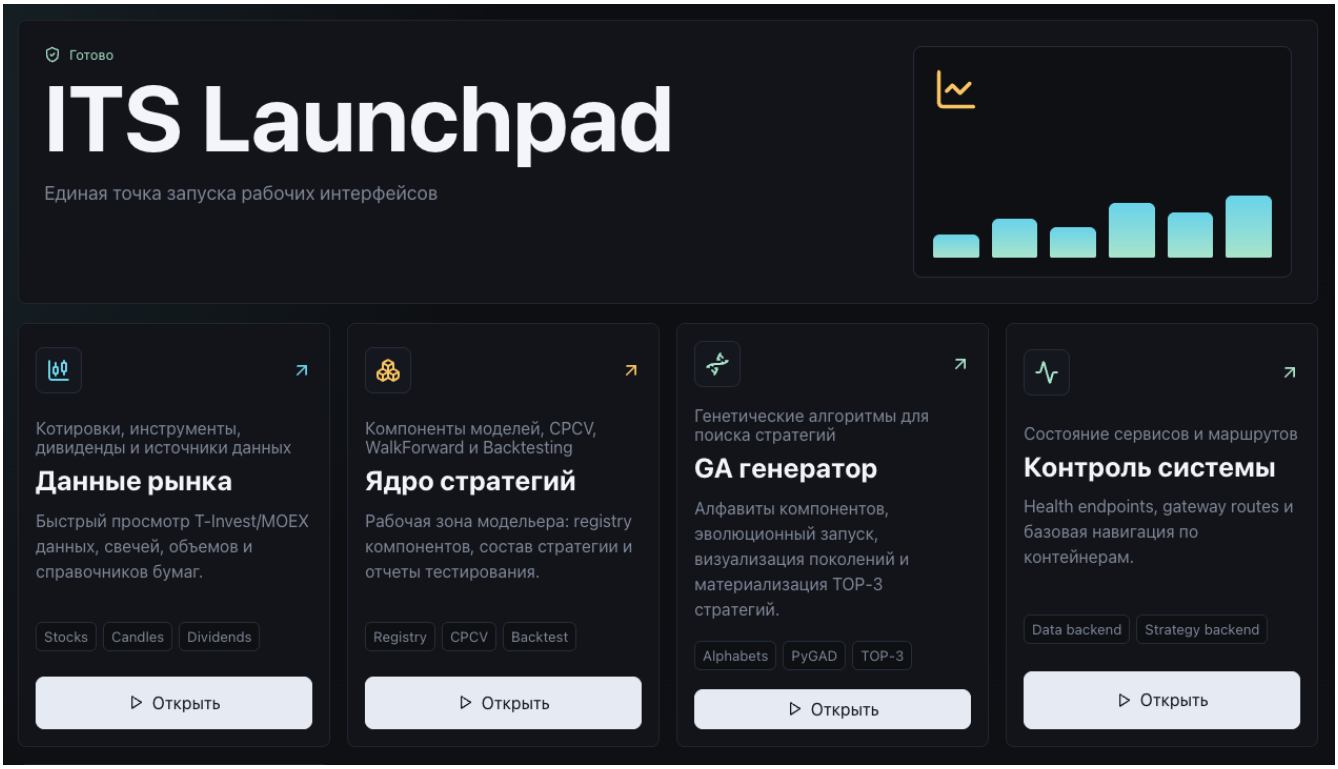
Contents

1. Contents

ITS Documentation

ITS (Intelligent Trading Strategies) is an intelligent system for designing, testing, comparing, and evolving trading strategies for exchange-traded markets, including equities, currencies, and crypto assets when the corresponding data adapters are added.

The primary user is a financial modeler, quant researcher, or analyst who designs trading models, validates their robustness on historical data, and assembles strategies from standardized components.



Documentation Map

Quick Start

1. Install Docker and Docker Compose v2.
2. Create a `.env` file in the repository root and set the T-Invest token:

```
tinvest_token=your_tinkoff_invest_token
```

1. Start the whole system with one command:

```
docker compose up --build
```

1. Open the main entry point:

http://localhost:8080/launchpad/

Main Interfaces

Interface	URL	Purpose
Launchpad	`/launchpad/`	Main screen for opening subsystems
Data Hub	`/data/`	Market data, instruments, quotes, dividends
Strategy Lab	`/strategies/`	Component registry, model assembly, testing
GA Lab	`/ga/`	Strategy generation with genetic algorithms
Data API	`/api/data/docs`	Data Backend Swagger
Strategy API	`/api/strategies/docs`	Strategy Backend Swagger
GA API	`/api/ga/docs`	GA Backend Swagger

What the User Gets

The system gives the modeler a single workspace for:

- connecting market data sources;
- exploring instruments, quotes, dividends, and derived bars;
- creating model components: `pre-selection`, `signal`, and `allocation`;
- assembling standardized strategies from those components;
- testing strategies with CPCV, WalkForward, and Backtesting;
- comparing strategies by an aggregate score;
- defining genetic-algorithm alphabets and generating new strategies automatically;
- materializing the best GA candidates as Python code in the model registry.

Current Implementation Status

The current version is a research and laboratory platform for forming, analyzing, testing, and generating trading strategies. It includes data infrastructure, UI, APIs, the strategy core, testing tools, and a GA generator. Production order execution, broker account management, risk-control services, and order-management modules can be implemented as separate extensions.

Purpose and User Roles

System Purpose

ITS is designed to develop trading strategies as a set of standardized and interchangeable components. The system separates the modeler's research work from infrastructure tasks such as data loading, return-matrix preparation, test execution, report rendering, model comparison, and automatic strategy generation.

The core product idea is to turn trading algorithm development into a controlled engineering workflow. The modeler describes the mathematical and economic logic of a component, while the platform handles:

- retrieving and normalizing market data;
- registering components;
- assembling strategies into a unified pipeline;
- running robustness tests;
- storing results;
- comparing models;
- searching and evolving strategies through a genetic algorithm.

Target Users

Financial Modeler

The main user. Typical tasks:

- formulate market-behavior hypotheses;
- create asset selectors, signal models, and allocators;
- validate strategies on historical data;
- compare results;
- pass the best strategies to further research or a production circuit.

Quant Researcher

Uses the system to research factor stability and portfolio rules:

- test different asset-selection regimes;
- assess stability on WalkForward windows;
- analyze overfitting risk through CPCV;
- compare return and risk metrics.

Integration Developer

Extends the system with new data sources:

- quotes;
- instrument reference data;
- dividends;
- alternative bars;

- fundamental, event, or news-based features.

Technical Buyer or Client

Receives the system as a platform that can be launched with one command, explored through UI, and extended through documented protocols.

Problems Solved by the System

1. Data Centralization

Data Hub connects market data sources and exposes a unified API for quotes, instruments, currencies, dividends, and custom bars.

2. Trading Model Standardization

A strategy is represented as a sequence of components:

```
pre-selection -> signal -> allocation
```

This structure makes models comparable, extensible, and suitable for automatic generation.

3. Fast Hypothesis Testing

Strategy Lab lets the user select a model, configure a test, and obtain:

- return and risk metrics;
- equity curve charts;
- WalkForward OOS curve;
- portfolio composition by rebalance date;
- sector aggregation;
- stop-loss and take-profit execution events for full trading strategies.

4. New Strategy Generation

GA Lab uses component alphabets as a search space. The genetic algorithm evaluates gene combinations and saves the best strategies to `its/strategies/models`.

5. LLM-Assisted Component Development

Because components have explicit protocols and narrow interfaces, LLMs can be used to rapidly generate new selectors, signals, and allocators. The modeler does not need to redesign the full testing pipeline for every new component.

Design Principles

- **standardized models** - each strategy has a unified and reproducible pipeline;
- **Open-Closed Principle** - new components can be added without changing the core;
- **Interface Segregation Principle** - selectors, signals, allocators, policies, and data sources use focused interfaces;
- **verifiability** - core behavior is covered by tests and available through APIs;
- **extensibility** - new data sources, components, and GA genes have dedicated directories;
- **reproducibility** - test results are cached and GA candidates are materialized as Python files.

User Outcomes

After working with the system, the user obtains:

- available asset and market data lists;
- assembled trading models;
- CPCV, WalkForward, and Backtesting reports;
- model rankings;
- generated strategy files;
- a software base for further scientific, product, or investment-analytical work.

Installation, Prerequisites, and Launch

Minimum Requirements

For regular use, the user needs:

- Docker Engine or Docker Desktop;
- Docker Compose v2;
- Git to obtain the source code;
- internet access for image builds and data-source requests;
- a T-Invest API token for market data;
- free port `8080`, or another port set through `ITS_GATEWAY_PORT`.

Recommended workstation resources:

- 4 CPU cores or more;
- 8 GB RAM or more;
- 5 GB of free space for images, caches, and test results.

Development Requirements

If the system is developed locally in addition to Docker launch:

- Python `3.12.x`;
- Poetry;
- Node.js `20+`;
- npm;
- a modern browser.

Main backend libraries:

- FastAPI and Uvicorn for API services;
- pandas and NumPy for data processing;
- scikit-learn pipeline interfaces for components;
- skfolio for portfolio optimization and model selection;
- vectorbt for backtesting;
- PyGAD for genetic algorithms;
- httpx, aiohttp, aiometer, async-lru for HTTP and asynchronous loading;
- t-tech-investments for T-Invest integration.

Main frontend libraries:

- Vue 3;
- Vite;
- TypeScript;
- lucide-vue-next;

- ECharts in Data UI.

Token Configuration

Create or update `.env` in the repository root:

```
tinvest_token=your_tinkoff_invest_token
```

Alternative names are also supported:

```
TINVEST_TOKEN=your_tinkoff_invest_token
TINKOFF_INVEST_API_TOKEN=your_tinkoff_invest_token
```

The token is required by Data Backend to retrieve instruments, quotes, currencies, and dividends from the T-Invest API.

Launching the Whole System

From the repository root:

```
docker compose up --build
```

This builds and starts:

- `data-backend`;
- `data-ui`;
- `strategy-backend`;
- `strategy-ui`;
- `ga-backend`;
- `ga-ui`;
- `launchpad-ui`;
- `nginx-gateway`.

URLs After Launch

Component	URL
Main screen	http://localhost:8080/launchpad/
Data Hub	http://localhost:8080/data/
Strategy Lab	http://localhost:8080/strategies/
GA Lab	http://localhost:8080/ga/
Documentation	http://localhost:8080/docs/
Data API Swagger	http://localhost:8080/api/data/docs
Strategy API Swagger	http://localhost:8080/api/strategies/docs
GA API Swagger	http://localhost:8080/api/ga/docs

If port `8080` is occupied:

```
ITS_GATEWAY_PORT=8090 docker compose up --build
```

The main screen will then be available at:

```
http://localhost:8090/launchpad/
```

Health Checks

After startup, open:

```
http://localhost:8080/health
```

Expected response:

```
ok
```

API health checks:

```
http://localhost:8080/api/data/health
http://localhost:8080/api/strategies/health
http://localhost:8080/api/ga/health
```

Expected JSON:

```
{"status":"ok"}
```

Data and Caches

Docker Compose creates named volumes:

Volume	Purpose
t-invest-cache	quote, reference-data, and dividend cache
strategy-test-cache	saved CPCV, WalkForward, and Backtesting reports
ga-cache	saved GA runs

GA also writes materialized strategies to:

```
its/strategies/models
```

Common Startup Issues

T-Invest Token Is Not Set

Data Backend returns 503 with a message that tinvest_token, TINVEST_TOKEN, or TINKOFF_INVEST_API_TOKEN must be configured.

Resolution: check .env and restart containers.

Port 8080 Is Occupied

Resolution:

```
ITS_GATEWAY_PORT=8090 docker compose up --build
```

No Quotes for the Selected Period

Possible causes:

- instrument not found;
- period is too early;
- the source returned no candles;
- invalid class_code or instrument type.

Resolution: check ticker, FIGI, class_code, interval, and dates in Data Hub.

GA Run Takes Too Long

Possible causes:

- large population;
- many generations;
- long data period;
- many assets;
- expensive CPCV or WalkForward settings.

Resolution: reduce `num_generations`, `sol_per_pop`, number of assets, or period length.

System Architecture

Overview

ITS consists of four user interfaces, three backend services, a Python strategy core, a data-loading subsystem, and a GA engine.

Docker Compose Containers

Service	Path	Purpose
nginx-gateway	infra/nginx	routes UI, API, and documentation
launchpad-ui	ui/launchpad-ui	system start screen
data-ui	ui/data-ui	data interface
strategy-ui	ui/strategy-ui	model and testing interface
ga-ui	its/ui/ga-ui	GA generation interface
data-backend	services/data_backend	data-source API
strategy-backend	services/strategy_backend	model registry and testing API
ga-backend	its/services/ga_backend	genetic-algorithm API

Gateway Routing

nginx-gateway exposes one external port and routes requests:

External path	Internal service
/	redirect to /launchpad/
/launchpad/	launchpad-ui
/data/	data-ui
/strategies/	strategy-ui
/ga/	ga-ui

External path	Internal service
/docs/	rendered Markdown documentation
/api/data/	data-backend
/api/strategies/	strategy-backend
/api/ga/	ga-backend

Backend Architecture

Data Backend

Path:

services/data_backend

Main responsibilities:

- health check;
- data-source list;
- stock reference data;
- currency reference data;
- candle loading;
- custom gold bar construction;
- dividend loading;
- response normalization and caching.

It uses data-loading code from:

its/data_loader

Strategy Backend

Path:

services/strategy_backend

Main responsibilities:

- component registry;
- core strategy model registry;
- full trading strategy registry;
- selected model details;
- CPCV execution and retrieval;
- WalkForward execution and retrieval;
- Backtesting execution and retrieval;
- latest-test model comparison.

GA Backend

Path:

its/services/ga_backend

Main responsibilities:

- read GA alphabets;
- start a GA job in the background;
- monitor run status;
- persist run history;
- materialize TOP-N strategies as Python code.

Frontend Architecture

All UIs are written in Vue 3, TypeScript, and Vite.

UI	Path	Role
Launchpad	ui/launchpad-ui	subsystem launch tiles
Data UI	ui/data-ui	market data workspace
Strategy UI	ui/strategy-ui	components, strategies, tests, comparison
GA UI	its/ui/ga-ui	genetic algorithm configuration and monitoring

The interfaces call APIs through the gateway, so the user does not need to know internal container addresses.

Strategy Code Core

Key directories:

Path	Purpose
its/strategies/core/selectors	pre-selection components
its/strategies/core/signals	signal models
its/strategies/core/optimization	portfolio allocators
its/strategies/core/types	base types and protocols
its/strategies/models	ready core strategy models
its/strategies_model/core	full trading strategy and exit policies
its/strategies_model/model	assembled trading strategies
its/strategies/testing	CPCV, WalkForward, Backtesting, comparison
its/ga/alphabets	GA gene alphabets
its/ga	registry, engine, materialization

Strategy Pipeline

The strategy core uses this pipeline:

```
pre-selection -> signal -> allocation
```

Each step can be replaced by a new component that follows the interface. This enables manual composition, automatic GA composition, and a unified testing circuit.

Inputs and Outputs

Inputs

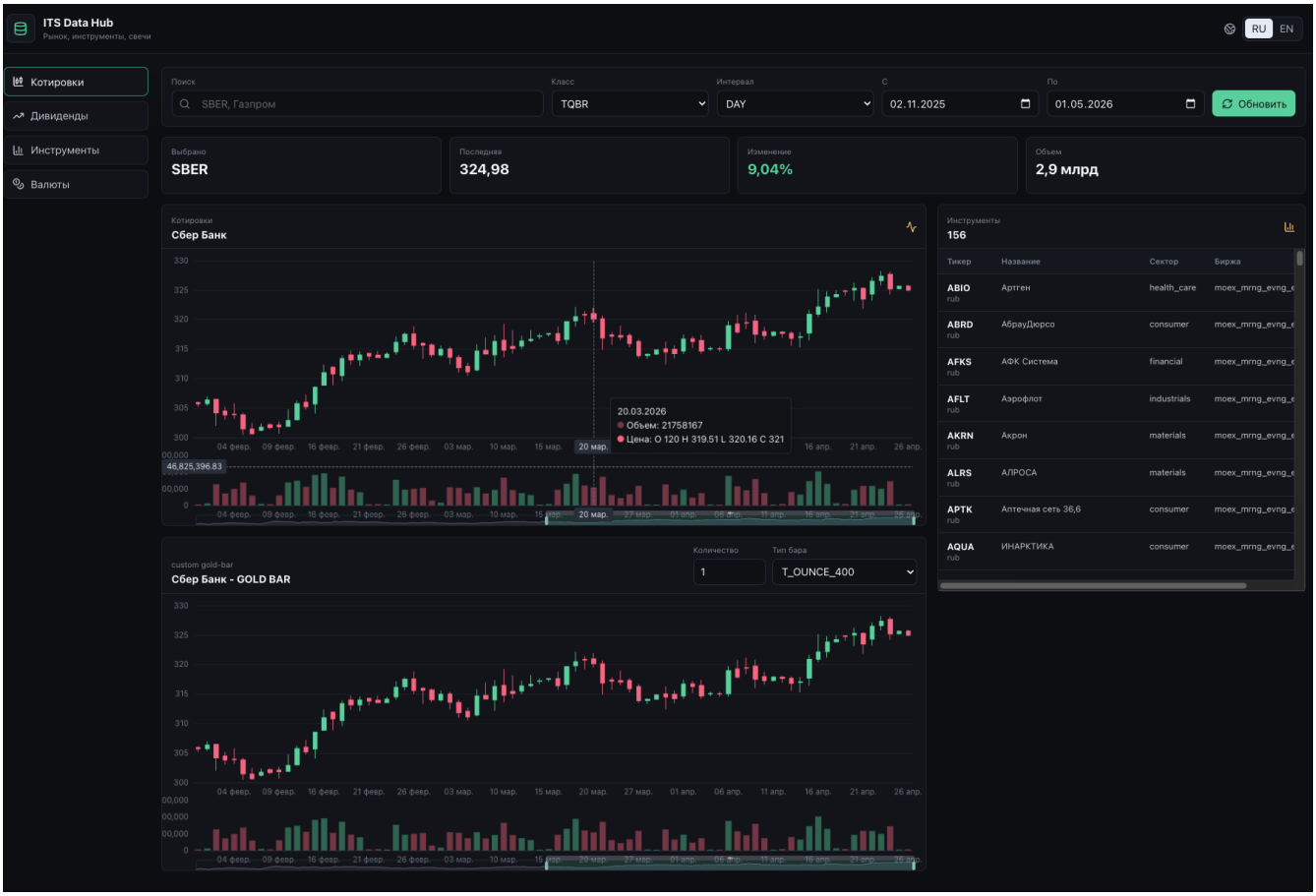
- instrument reference data;
- historical OHLCV candles;
- dividends;
- testing parameters;
- model classes;
- GA alphabets.

Outputs

- cleaned data tables;
- UI charts and tables;
- JSON reports for CPCV, WalkForward, and Backtesting;
- aggregate model ranking;
- materialized Python classes for generated strategies;
- data and test caches.

Data Hub

Data Hub is the ITS data subsystem. It connects sources, retrieves market information, normalizes data, and provides a unified API for other services.



Purpose

Data Hub solves three groups of tasks:

- gives the modeler a visual interface for data exploration;
- provides backend API for Strategy Lab and GA Lab;
- isolates external source integrations from strategy logic.

Main Files

Path	Purpose
services/data_backend/app/main.py	Data Backend FastAPI application
ui/data-ui	Data Hub Vue interface
its/data_loader	data loaders and source adapters
its/data_loader/t_invest_data_readers	connected T-Invest source
its/data_loader/custom_bar/gold_bar	custom gold bar construction

Data Available in the Base Version

The base version connects T-Invest and provides:

- Russian equities;
- currency instruments;
- OHLCV candles;
- ticker reference data;
- sectors, exchanges, currencies, instrument classes;
- dividends;
- custom gold bars recalculated through gold.

UI Sections

Quotes

Used to inspect candles for a selected instrument.

The user selects:

- instrument class, for example `TQBR`;
- candle interval;
- start date;
- end date;
- instrument.

The UI displays:

- candlestick chart;
- latest price;
- percentage change for the period;
- number of candles;
- volume.

Dividends

Shows dividend history for the selected security:

- declaration date;
- last buy date;
- record date;
- payment date;
- dividend amount;
- yield;
- type and regularity.

Dividends are also used as input data for components such as `DividendHistorySelector`.

Instruments

Stock reference section. The user can search by name or ticker and see:

- FIGI;
- ticker;
- ISIN;
- name;
- sector;
- exchange;
- currency;
- country of risk;
- lot;
- trading status;
- buy/sell availability.

Currencies

Currency instrument section. It is also used for `GLDRUB_TOM`, which is required for custom gold bars.

Custom Gold Bar

Data Hub can build alternative bars through gold value. The user sets:

- base instrument;
- gold instrument, default `GLDRUB_TOM`;
- number of gold units;
- bar type, for example `GRAM`, `T_OUNCE`, `KG`, `T_OUNCE_400`;
- period and interval.

The result is OHLCV bars recalculated through the value of the selected gold amount.

Data Backend API

External requests go through the gateway:

`/api/data/`

Main endpoints:

Endpoint	Purpose
`GET /api/data/health`	status check
`GET /api/data/sources`	source list
`GET /api/data/stocks`	stock list
`GET /api/data/currencies`	currency list
`GET /api/data/prices`	OHLCV candles
`GET /api/data/custom-gold-bars`	custom gold bars
`GET /api/data/dividends`	dividends

Endpoint	Purpose
GET /api/data/docs	Swagger

Supported Candle Intervals

- CANDLE_INTERVAL_1_MIN;
- CANDLE_INTERVAL_5_MIN;
- CANDLE_INTERVAL_15_MIN;
- CANDLE_INTERVAL_HOUR;
- CANDLE_INTERVAL_DAY;
- CANDLE_INTERVAL_WEEK;
- CANDLE_INTERVAL_MONTH.

Caching

T-Invest loaders use a file cache inside its/data in the container. Docker Compose maps it to the volume:

t-invest-cache

The cache reduces external API calls, speeds up repeated tests, and reuses already-loaded periods.

Adding a New Data Source

New integrations are added under:

its/data_loader

Recommended workflow:

1. Create a source adapter.
2. Normalize output fields to the format expected by Data Backend.
3. Add a use case if dedicated business logic is required.
4. Extend Data Backend endpoints or add a new one.
5. Update Data UI if the data must be visually available.

Data Requirements for Strategies

Strategies and tests expect candles to contain at least:

Field	Purpose
time	candle date and time
ticker	instrument ticker
figi	FIGI
open	open price
high	high price
low	low price
close	close price

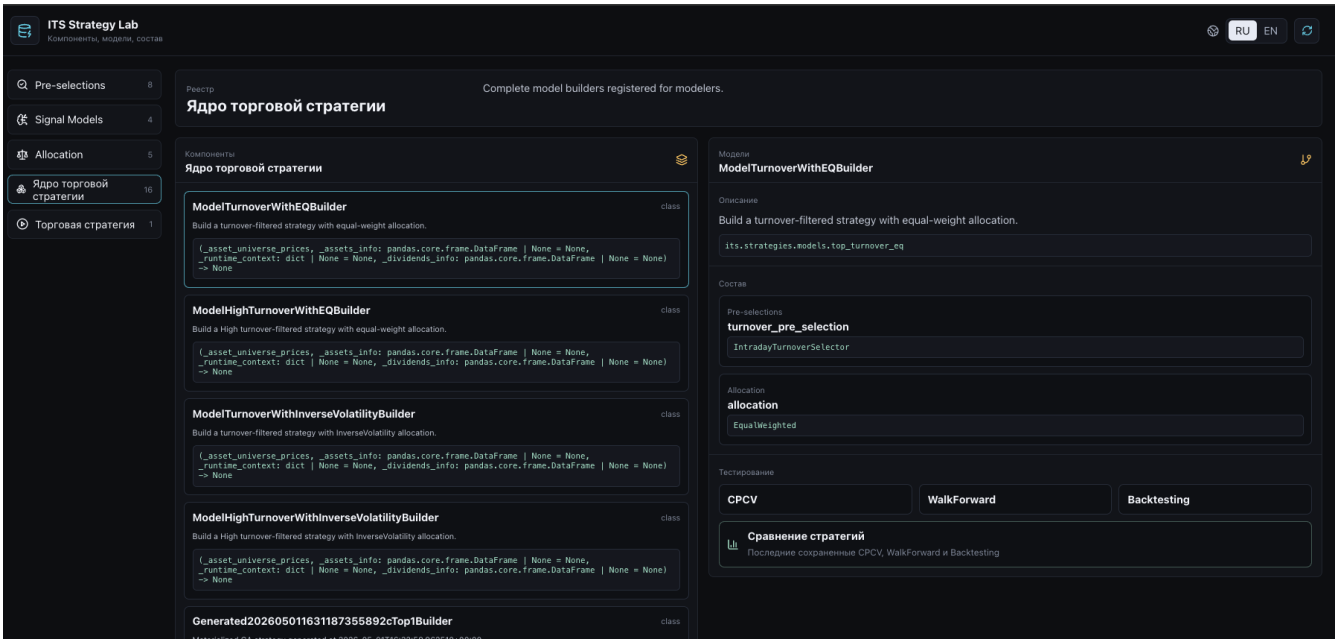
Field	Purpose
volume	volume
is_complete	completed-candle flag

Dividend selectors require:

Field	Purpose
ticker	ticker
last_buy_date	last buy date to receive the dividend

Strategy Lab

Strategy Lab is the main workspace for the financial modeler. It displays components, ready models, strategy composition, CPCV, WalkForward, Backtesting, and model comparison.



Purpose

Strategy Lab is used to:

- inspect registered components;
- select ready models;
- understand model composition;
- run tests;
- read saved results;
- compare strategies;
- analyze portfolio composition and sector exposure.

Main Files

Path	Purpose
services/strategy_backend	Strategy Lab FastAPI backend
ui/strategy-ui	Strategy Lab Vue UI
its/strategies/core	strategy-core components
its/strategies/models	ready core models
its/strategies/testing	testing methods
its/strategies_model	full trading strategies

Strategy Structure

The strategy core consists of three sequential steps:

```
pre-selection -> signal -> allocation
```

Pre-selection

Initial asset filtering. The selector receives a price or return matrix and produces a mask of assets allowed for further processing.

Current examples:

- `IntradayTurnoverSelector` - turnover-based selection;
- `CrossSectionalMomentumSelector` - cross-sectional momentum selection;
- `DividendHistorySelector` - dividend-history selection;
- `SectorSelector` - sector-based selection;
- `TrendSelector` and `TrendThresholdSelector` - trend-based selection;
- `KeepAllSelector` - pass-through selection;
- `SafeEmptySelector` - wrapper that protects the pipeline from empty selection.

Signal

The signal layer applies additional selection logic after pre-selection.

Examples:

- `KeepAllSignal` - passes all assets;
- `PriceBreakoutSignal` - selects assets breaking above a recent high;
- `SMACrossSignal` - moving-average cross signal;
- `TwoCandlePositiveTrendSignal` - positive movement over recent candles.

Allocation

The allocator assigns portfolio weights to selected assets.

Examples:

- `EqualWeighted`;
- `InverseVolatility`;
- `HierarchicalRiskParity`;
- `CVaR`;
- `CVaRHighRisk`.

Allocators use skfolio and custom extensions.

Strategy Lab Tabs

Pre-selections

Displays registered selectors. Each item shows:

- class name;
- module;

- description;
- constructor signature;
- parameters;
- source path.

Signal Models

Displays registered signal models with the same metadata.

Allocation

Displays registered allocators.

Core Strategy

The key tab. It displays models from:

its/strategies/models

For a selected model the user sees:

- name;
- description;
- pipeline composition;
- available tests;
- saved report list;
- buttons to run CPCV, WalkForward, and Backtesting.

Trading Strategy

A full trading strategy includes:

- portfolio-model core;
- entry and exit rules;
- stop-loss;
- take-profit;
- additional position lifecycle policies.

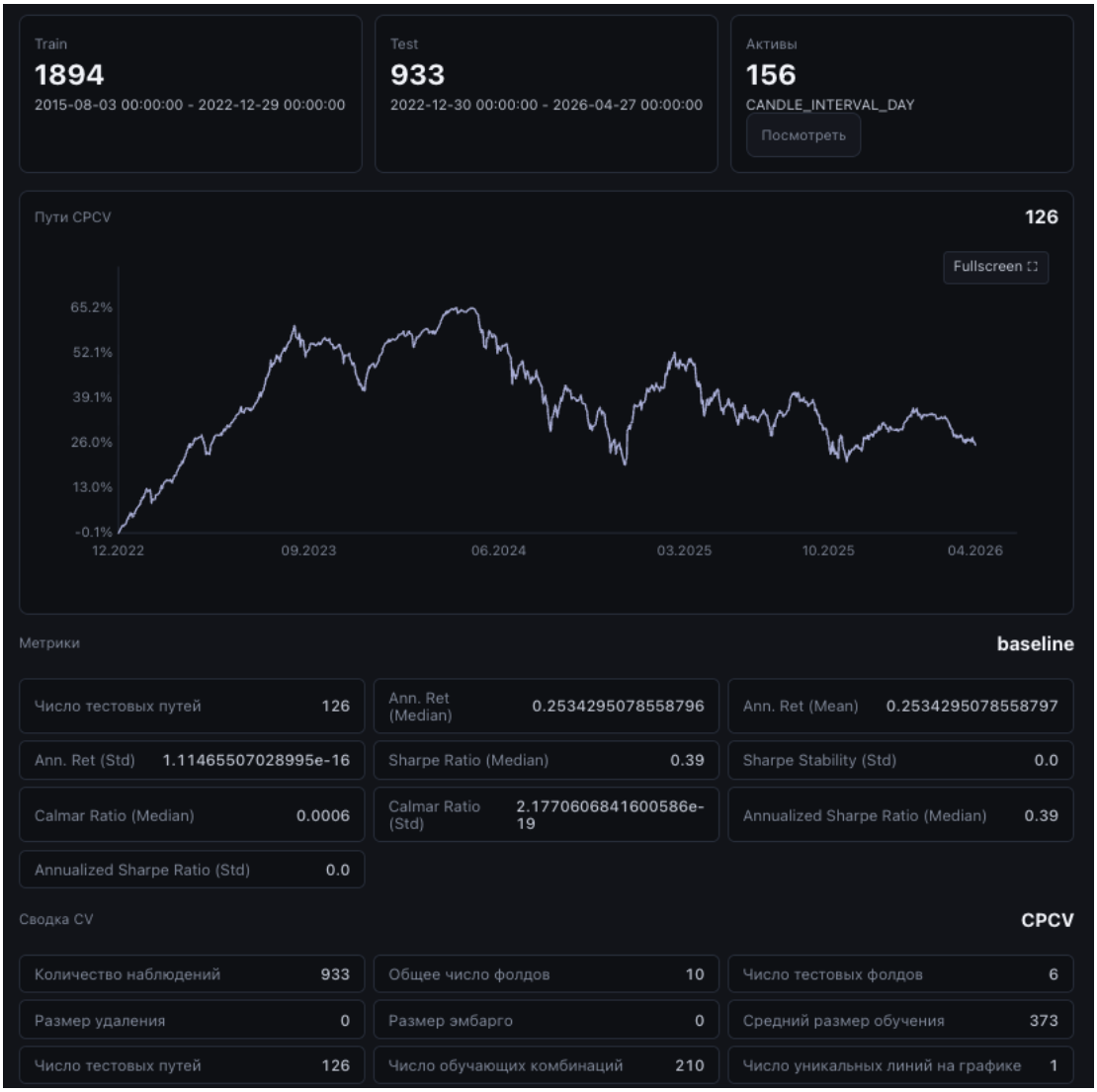
Current example:

TurnoverEqStopLoss1TakeProfit3Builder

It uses `ModelTurnoverWithEQBuilder`, `1%` stop-loss, and `3%` take-profit.

CPCV

CPCV opens in a separate modal window.



The user sets:

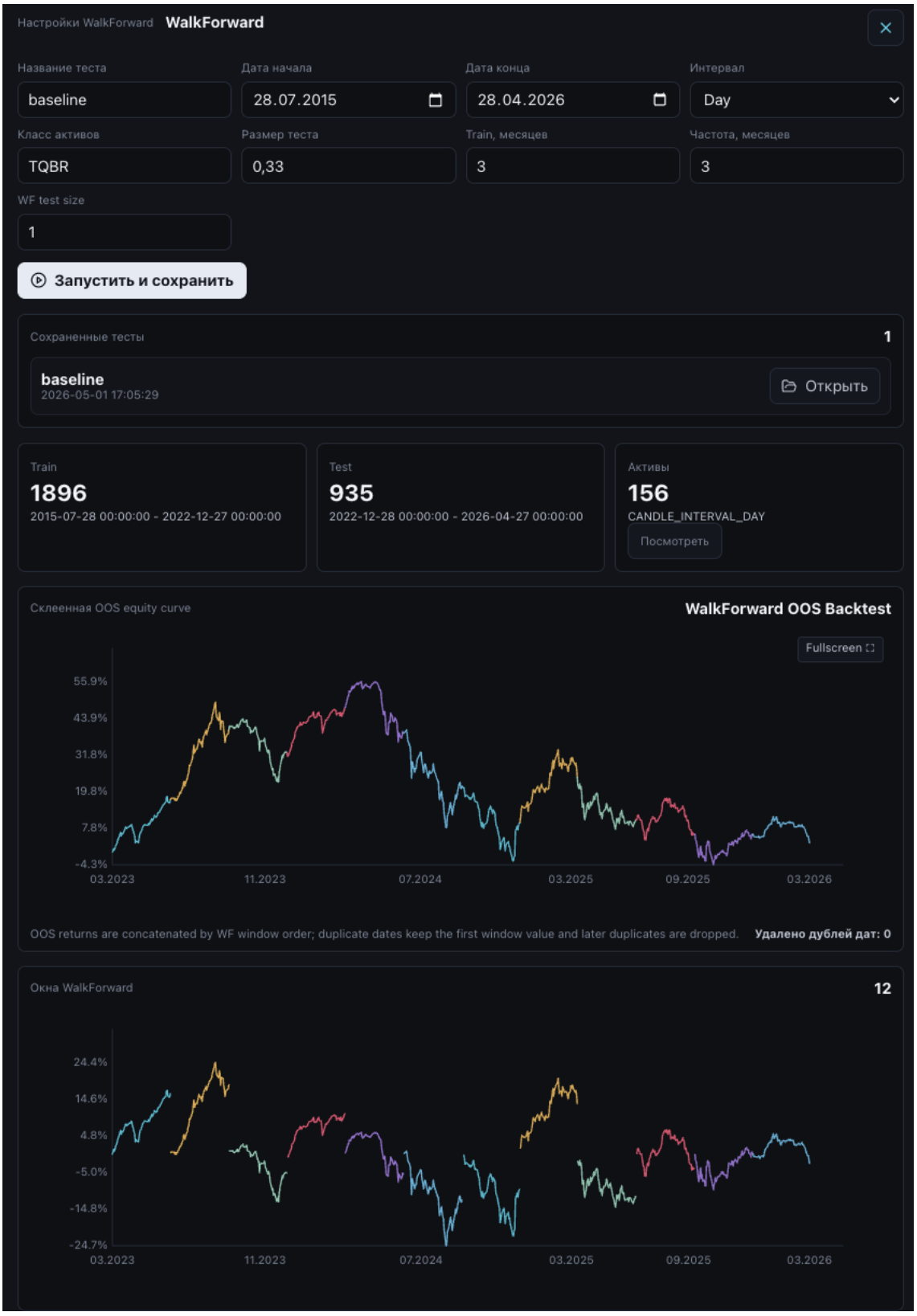
- test name;
- start date;
- end date;
- interval;
- asset class;
- `n_folds`;
- `n_test_folds`.

Result:

- CPCV summary;
- metrics by test paths;
- cumulative-return path charts;
- asset list;
- saved JSON report.

WalkForward

WalkForward opens in a separate modal window.



The user sets:

- test name;
- data period;

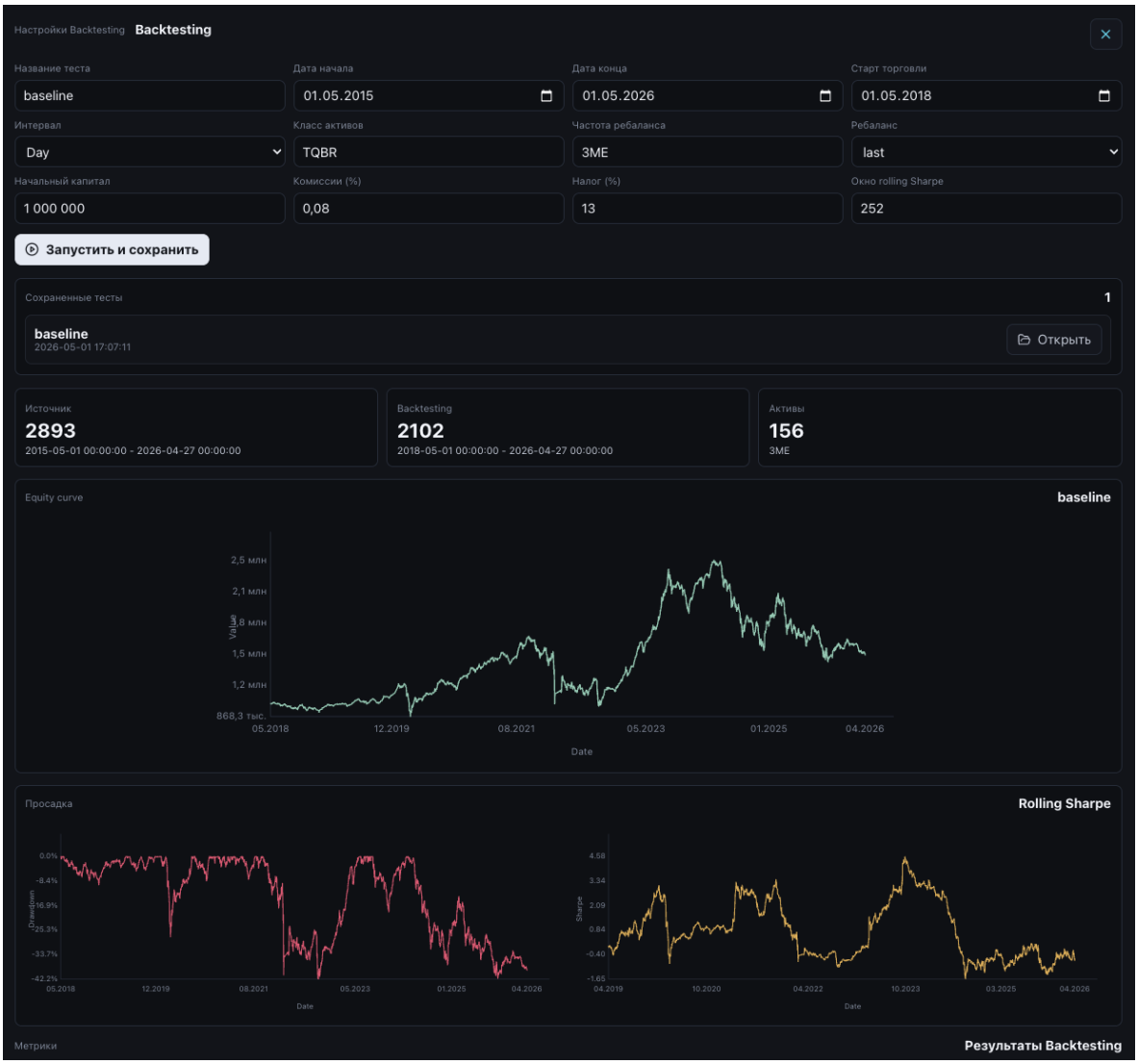
- OOS test fraction;
- train window size in months;
- window shift frequency;
- WF test size;
- asset class and interval.

Result:

- WalkForward windows;
- metrics;
- individual window charts;
- stitched OOS equity curve;
- number of removed duplicate dates;
- saved JSON report.

Backtesting

Backtesting opens in a separate modal window.



The user sets:

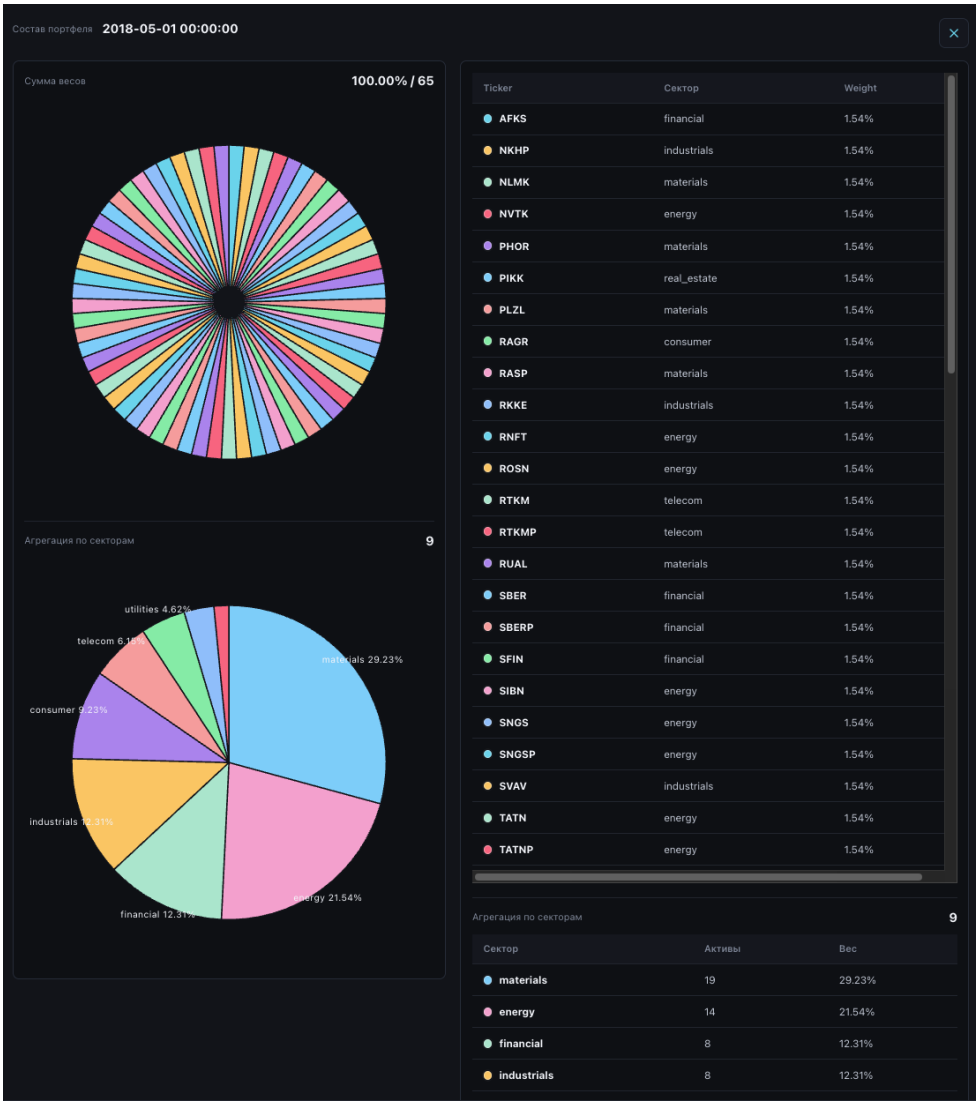
- data period;
- trading start date;
- rebalance frequency;
- rebalance date rule;
- initial capital;
- fees;
- slippage;
- tax rate;
- rolling Sharpe window.

Result:

- equity curve;
- drawdown curve;
- rolling Sharpe;
- vectorbt metrics table;
- return and risk summary;
- portfolio weights at rebalances;
- stop-loss and take-profit events for full trading strategies.

Portfolio Composition

Backtesting results include a portfolio-composition modal.

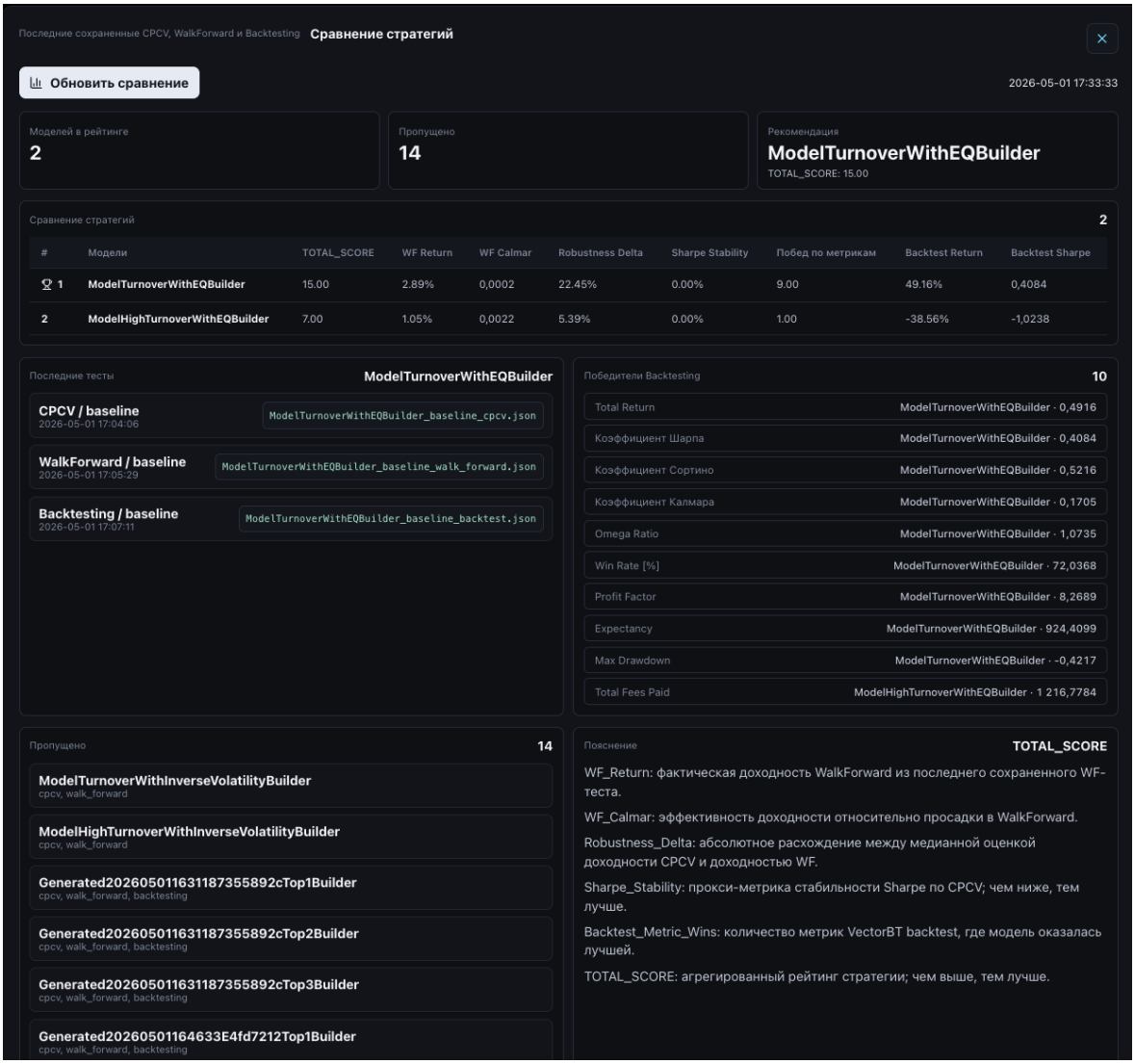


Available details:

- securities included at each rebalance;
- each security weight;
- total weights;
- sector aggregation;
- sector exposure chart.

Model Comparison

Model comparison opens in a separate modal.



The system takes the latest saved CPCV, WalkForward, and Backtesting results for each model and builds an aggregate ranking.

Metrics used:

- `WF_Return`;
- `WF_Calmar`;
- `Robustness_Delta`;
- `Sharpe_Stability`;
- Backtesting metric wins;
- `TOTAL_SCORE`.

A model enters comparison only if a full set of saved tests is available.

Saved Results

Test results are saved as JSON caches:

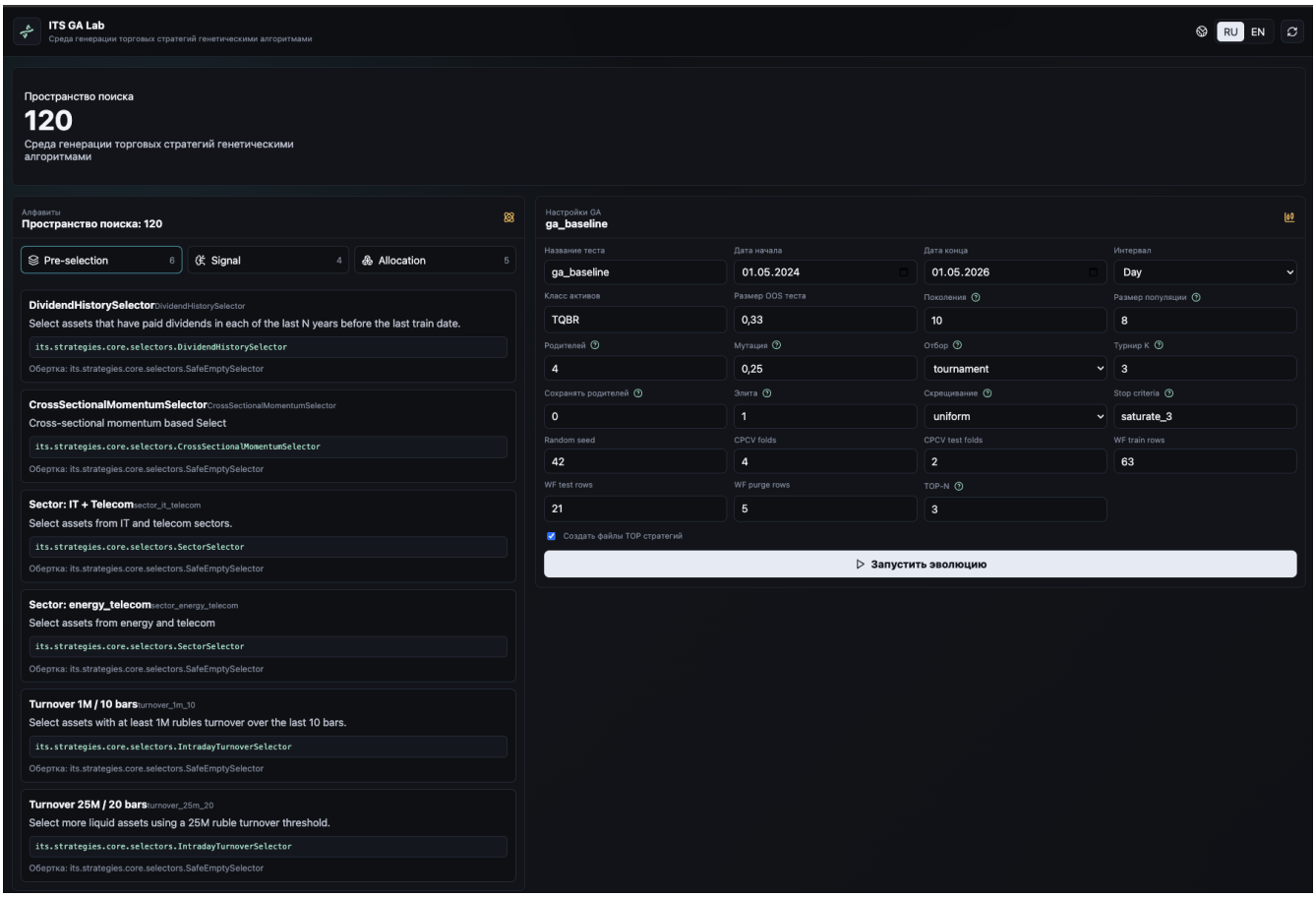
Type	Environment variable	Container path
CPCV	`STRATEGY_TEST_CACHE_DIR`	`/app/its/data/strategy_tests/cpcv`

Type	Environment variable	Container path
WalkForward	`STRATEGY_WF_CACHE_DIR`	`/app/its/data/strategy_tests/walk_forward`
Backtesting	`STRATEGY_BACKTEST_CACHE_DIR`	`/app/its/data/strategy_tests/backtest`

Docker Compose persists them in `strategy-test-cache`.

GA Lab

GA Lab is the subsystem for automatic trading-strategy generation using genetic algorithms.



Purpose

GA Lab uses already-created system components as gene alphabets:

pre-selection gene + signal gene + allocation gene = candidate strategy

The genetic algorithm explores combinations, evaluates them on historical data, and saves the best strategies into the codebase.

Main Files

Path	Purpose
its/services/ga_backend	GA FastAPI backend
its/ui/ga-ui	GA Lab Vue UI
its/ga/engine.py	main GA engine
its/ga/registry.py	alphabet loading
its/ga/types.py	GeneDefinition description
its/ga/materialization.py	Python strategy-file generation

Path	Purpose
its/ga/alphabets	gene alphabets
its/strategies/models	destination for TOP strategies

Alphabets

The system has three gene groups.

Pre-selection

Path:

its/ga/alphabets/pre_selection

Current examples:

- DividendHistorySelector;
- CrossSectionalMomentumSelector;
- sector_it_telecom;
- sector_energy_telecom;
- turnover_1m_10;
- turnover_25m_20.

Signal

Path:

its/ga/alphabets/signal

Current examples:

- pass_signal;
- PriceBreakoutSignal;
- SMACrossSignal;
- TwoCandlePositiveTrendSignal.

Allocation

Path:

its/ga/alphabets/allocation

Current examples:

- equal_weighted;
- inverse_volatility;
- HierarchicalRiskParity;
- CVaR;
- CVaRHighRisk.

Search Space

The search-space size is the product of gene counts:

N(pre-selection) * N(signal) * N(allocation)

Each candidate is encoded by three indexes:

```
(selector_idx, signal_idx, allocation_idx)
```

After decoding, the candidate gets a name:

```
[GA][selector_name][signal_name][allocation_name]
```

GA Settings

UI settings:

Parameter	Purpose
test_name	run name
start_date, end_date	data period
interval	candle interval
class_code	asset class, for example TQBR
test_size	out-of-sample fraction
num_generations	number of generations
sol_per_pop	population size
num_parents_mating	number of parents
mutation_probability	mutation probability
parent_selection_type	parent selection method
k_tournament	tournament size
keep_parents	parent preservation
keep_elitism	number of elite solutions
crossover_type	crossover type
mutation_type	mutation type
stop_criteria	PyGAD early stop
random_seed	reproducibility
cpcv_n_folds	CPCV folds inside evaluation
cpcv_n_test_folds	CPCV test folds
wf_train_size	WalkForward train rows
wf_test_size	WalkForward test rows
wf_purged_size	purge rows
top_n	number of best strategies to return
materialize_top	create files for TOP strategies

Candidate Evaluation

Each candidate strategy is built as:

```
ga_pre_selection -> ga_signal -> ga_allocation
```

The GA engine:

1. Loads data through Data Backend.
2. Builds the return matrix.
3. Splits the period into train and test.
4. Fits the pipeline on train.
5. Evaluates the candidate on test through CPCV and WalkForward.
6. Extracts metrics.
7. Computes `TOTAL_SCORE`.

Fitness Metrics

Current metrics:

Metric	Direction	Weight
`WF_Return`	higher is better	`0.30`
`WF_Calmar`	higher is better	`0.30`
`Robustness_Delta`	lower is better	`0.20`
`Sharpe_Stability`	lower is better	`0.20`

Penalties are also applied:

- for max drawdown above the threshold;
- for overly narrow portfolios with too few active assets.

Final scale:

```
TOTAL_SCORE = 100 * weighted_score - penalties
```

Run Results

GA Lab displays:

- run status;
- generations;
- best score;
- average score;
- best candidate;
- population;
- TOP strategies;
- materialized files;
- materialization errors if any.

Strategy Materialization

If `materialize_top` is enabled, TOP-N strategies are saved to:

```
its/strategies/models
```

Each candidate creates a Python file:

`ga_generated_<run_id>_top_<rank>.py`

The file:

`its/strategies/models/__init__.py`

is also updated. The generated strategy then appears in Strategy Lab as a regular registered model.

Run Caches

GA runs are saved as JSON under:

`/app/its/data/ga_runs`

Docker Compose persists them in:

`ga-cache`

Practical Workflow

1. The modeler creates selectors, signals, and allocators.
2. Adds them to GA alphabets.
3. Opens GA Lab.
4. Configures the period, population size, and generation count.
5. Runs evolution.
6. Reviews TOP strategies.
7. Materializes the best strategies.
8. Opens Strategy Lab.
9. Runs full CPCV, WalkForward, and Backtesting.
10. Compares new models with existing ones.

Modeler and Component Developer Guide

This section describes how to extend the system with new data sources, model components, ready strategies, and GA genes.

General Principle

The modeler should describe only the domain logic:

- how to select assets;
- how to form a signal;
- how to allocate weights;
- how to manage a position.

The platform handles infrastructure:

- data loading;
- matrix preparation;
- test execution;
- report construction;
- UI registration;
- result comparison;
- generation of new combinations through GA.

Component Types

Selector

A selector performs preliminary asset filtering.

Base type:

its/strategies/core/types/selectors_types.py

The class should inherit from Selectros and implement fit. Inside fit, set:

self.to_keep_ = ...

to_keep_ is a boolean asset mask.

Signal

A signal model performs additional selection after pre-selection.

Base type:

its/strategies/core/types/signals_types.py

The class inherits from Siglans and also sets to_keep_.

Allocation

An allocator defines portfolio weights. Current implementation uses skfolio allocators and custom extensions:

its/strategies/core/optimization

An allocator should be compatible with the pipeline and produce weights through the skfolio interface.

Creating a New Selector

1. Create a file under:

```
its/strategies/core/selectors
```

1. Implement the class.

Minimal structure:

```
import numpy as np
import sklearn.utils.validation as skv

from its.strategies.core.types.selectors_types import Selectros


class MySelector(Selectros):
    """Selector logic description."""

    def __init__(self, my_parameter: int = 10):
        self.my_parameter = my_parameter

    def fit(self, X, y=None):
        X = skv.validate_data(self, X, ensure_all_finite="allow-nan")
        self.to_keep_ = np.ones(X.shape[1], dtype=bool)
        return self
```

1. Add the class to:

```
its/strategies/core/selectors/__init__.py
```

1. Add the class name to `__all__`.

After that, Strategy Backend can display the component in the registry.

Creating a New Signal

1. Create a file under:

```
its/strategies/core/signals
```

1. Inherit from `Siglans`.
2. Implement `fit`.
3. Add import and `__all__` entry in:

```
its/strategies/core/signals/__init__.py
```

Creating a New Allocator

1. Create or extend a file under:

```
its/strategies/core/optimization
```

1. If skfolio is used, keep compatibility with `BaseOptimization`.
2. Export the allocator from:

```
its/strategies/core/optimization/__init__.py
```

Creating a Core Strategy Model

Ready models are stored in:

```
its/strategies/models
```

A model should inherit from `StrategyBuilder`.

Example:

```
from typing import override
```

```
from its.strategies.core.optimization import EqualWeighted
from its.strategies.core.selectors import IntradayTurnoverSelector
from its.strategies.core.types.strategy_types import Pipeline, Strategy, StrategyBuilder


class MyStrategyBuilder(StrategyBuilder):
    """Short strategy description."""

    @override
    def build(self) -> Strategy:
        return Strategy(
            name="MyStrategy",
            description="Model description",
            pipeline=Pipeline(
                steps=[
                    (
                        "pre_selection",
                        IntradayTurnoverSelector(
                            asset_universe_prices=self._asset_universe_prices,
                            lookback_bars=10,
                            min_turnover=1_000_000,
                        ),
                    ),
                    ("allocation", EqualWeighted()),
                ]
            ),
        )
```

Registration:

1. Import the class in its/strategies/models/__init__.py.
2. Add the class name to __all__.

The model will then appear in the Core Strategy tab.

Creating a Full Trading Strategy

A full trading strategy is one level above the core: it adds position lifecycle rules.

Base types:

its/strategies_model/core/trading_strategy.py

Inherit from TradingStrategyBuilder and implement:

- name;
- description;
- build_core_strategy;
- optionally build_exit_policy;
- optionally build_metadata.

Example:

```
from typing import override

from its.strategies.models.top_turnover_eq import ModelTurnoverWithEQBuilder
from its.strategies_model.core.trading_strategy import (
    FixedStopTakeProfitPolicy,
    TradingStrategyBuilder,
)


class MyTradingStrategyBuilder(TradingStrategyBuilder):
    @property
    @override
    def name(self) -> str:
        return "MyTradingStrategy"

    @property
```

```
    @override
    def description(self) -> str:
        return "Core strategy with fixed stop loss and take profit."

    @override
    def build_core_strategy(self):
        return ModelTurnoverWithEQBuilder(
            self._asset_universe_prices,
            self._assets_info,
            self._runtime_context,
            self._dividends_info,
        ).build()

    @override
    def build_exit_policy(self):
        return FixedStopTakeProfitPolicy(
            stop_loss_pct=0.01,
            take_profit_pct=0.03,
        )
```

Register it in:

```
its/strategies_model/model/__init__.py
```

Adding a Component to GA

GA uses `GeneDefinition`.

File:

```
its/ga/types.py
```

Minimal gene example:

```
from its.ga.types import GeneDefinition
from its.strategies.core.signals import KeepAllSignal

GENES = [
    GeneDefinition(
        id="pass_signal",
        title="Pass-signal",
        description="Do not apply an additional signal filter.",
        group="signal",
        component=KeepAllSignal,
    )
]
```

Groups:

```
pre_selection
signal
allocation
```

Genes are placed in:

```
its/ga/alphabets/pre_selection
its/ga/alphabets/signal
its/ga/alphabets/allocation
```

If a component needs runtime data, use `runtime_args`:

```
GeneDefinition(
    id="DividendHistorySelector",
    title="DividendHistorySelector",
    description="Select assets with dividend history.",
    group="pre_selection",
    component=DividendHistorySelector,
    wrapper=SafeEmptySelector,
    runtime_args={"dividends_df": "dividends_info"},
)
```

GA Runtime Context

GA passes:

Key	Content
`asset_universe_prices`	long candle table
`assets_info`	asset reference table
`dividends_info`	dividends

Component Recommendations

- Do not use future data when calculating features.
- Validate inputs and return clear errors.
- Use `SafeEmptySelector` if the component can select zero assets.
- Add docstrings: Strategy UI shows descriptions from code.
- Make parameters explicit in `__init__`.
- Do not change global state.
- Keep compatibility with pandas DataFrame and sklearn/skfolio pipeline.
- Cover new logic with tests.

Testing New Components

Tests are located under:

```
tests
```

Recommended commands:

```
poetry run pytest
poetry run ruff check .
poetry run mypy .
```

The current project includes tests for data loaders, selectors, signals, allocators, strategies, GA engine, and trading strategy backtesting.

Testing and Model Comparison Methodology

ITS contains three main model-validation methods:

1. CPCV.
2. WalkForward.
3. Backtesting.

It also implements aggregate model comparison by the latest saved tests.

General Data Flow

```
flowchart TD
    Model["Selected model"] --> Data["Data Backend<br/>stocks + prices + dividends"]
    Data --> Matrix["Return matrix or close prices"]
    Matrix --> Fit["Fit pipeline"]
    Fit --> Test["CPCV / WalkForward / Backtesting"]
    Test --> Report["JSON report"]
    Report --> UI["Strategy UI charts and tables"]
    Report --> Cache["Test cache"]
```

Data Preparation

CPCV and WalkForward use a return matrix:

1. Use `time`, `ticker`, and `close`.
2. Pivot data into a wide matrix: rows are dates, columns are tickers.
3. Forward-fill missing prices.
4. Remove assets with fully empty or constant prices.
5. Compute `pct_change`.
6. Fill the initial missing return with zero.

Backtesting uses close prices. Full trading strategies also use high/low prices for stop-loss and take-profit processing.

CPCV

CPCV means Combinatorial Purged Cross-Validation. It is used to assess strategy robustness and reduce overfitting risk.

The system uses:

```
skfolio.model_selection.CombinatorialPurgedCV
```

Parameters:

Parameter	Meaning
`n_folds`	number of folds
`n_test_folds`	number of test folds per combination
`start_date`, `end_date`	data period
`interval`	candle interval
`class_code`	asset class

CPCV shows:

- result dispersion across paths;
- median return;
- average return;
- return standard deviation;
- Sharpe stability;
- number of test paths;
- cumulative-return charts by path.

Use it:

- for initial validation of a new model;
- when estimating sensitivity to data splitting;
- before more expensive WalkForward or Backtesting runs.

WalkForward

WalkForward tests a strategy on sequential time windows.

The system uses:

skfolio.model_selection.WalkForward

Parameters:

Parameter	Meaning
test_size	fraction of OOS period after initial train split
train_size_months	train-window size
freq_months	window shift frequency
wf_test_size	test-segment size

WalkForward shows:

- train/test windows;
- OOS-period metrics;
- separate window curves;
- stitched OOS equity curve;
- final OOS return.

Use it:

- to validate behavior over time;
- to evaluate stability across market regimes;
- to select strategies before Backtesting.

Backtesting

Backtesting simulates historical trading with rebalances, fees, slippage, and initial capital.

The system uses:

vectorbt

Parameters:

Parameter	Meaning
trading_start_date	trading start date
rebalance_freq	rebalance frequency, for example 3ME
rebalance_on	rebalance date rule
init_cash	initial capital
fees	fees
slippage	slippage
tax_rate	tax rate for after-tax estimate
rolling_window	rolling Sharpe window

Backtesting returns:

- total return;
- after-tax return;
- maximum drawdown;
- maximum drawdown duration;
- equity curve;
- drawdown curve;
- rolling Sharpe;
- vectorbt stats table;
- portfolio weights at each rebalance;
- portfolio composition;
- sector structure;
- stop-loss and take-profit events.

Full Trading Strategy Testing

For models from its/strategies_model/model, Backtesting additionally considers:

- high/low prices;
- exit policies;
- FixedStopTakeProfitPolicy;
- stop_loss and take_profit events;
- entry price;
- execution price;
- closed-position return.

Strategy Comparison

Comparison endpoint:

GET /api/strategies/comparison/latest

The system uses latest saved:

- CPCV;
- WalkForward;
- Backtesting.

A model is skipped if it does not have the full test set.

Comparison Metrics

Metric	Source	Interpretation
WF_Return	WalkForward	realized OOS return
WF_Calmar	WalkForward	return relative to drawdown
Robustness_Delta	CPCV + WF	gap between CPCV median return and WF return
Sharpe_Stability	CPCV	Sharpe stability proxy
Backtest_Metric_Wins	Backtesting	number of backtest metric wins
TOTAL_SCORE	comparison	final ranking

Backtesting Metrics Used for Winners

Higher is better:

- Total Return;
- Sharpe Ratio;
- Sortino Ratio;
- Calmar Ratio;
- Omega Ratio;
- Win Rate [%];
- Profit Factor;
- Expectancy.

Lower is better:

- Max Drawdown;
- Total Fees Paid.

Practical Interpretation

A strategy is more promising when:

- WalkForward shows positive OOS return;
- Calmar Ratio is higher than alternatives;
- the gap between CPCV and WalkForward is small;

- CPCV dispersion is moderate;
- Backtesting does not show excessive drawdown;
- the portfolio is not concentrated in too few assets;
- results do not depend on one lucky period.

Limitations

Test results are not a guarantee of future performance. Historical validation reflects only the available data and depends on data quality, selected parameters, fees, liquidity, and market regimes.

API and Integrations

All external requests go through `nginx-gateway`. Paths below are available after launch on `localhost:8080`.

Swagger

API	URL
Data Backend	`/api/data/docs`
Strategy Backend	`/api/strategies/docs`
GA Backend	`/api/ga/docs`

Data API

Base path:

```
/api/data/
```

Health

```
GET /api/data/health
```

Response:

```
{"status":"ok"}
```

Sources

```
GET /api/data/sources
```

Returns connected sources and available resources.

Stocks

```
GET /api/data/stocks
```

Main parameters:

Parameter	Description
`class_code`	instrument class, default `TQBR`
`search`	ticker or name search
`tickers`	ticker list
`exchange`	exchange filter
`sector`	sector filter
`country_of_risk`	country of risk
`limit`, `offset`	pagination

Currencies

```
GET /api/data/currencies
```

Returns currency instruments.

Prices

```
GET /api/data/prices
```

Main parameters:

Parameter	Description
figis	FIGI list
tickers	ticker list
class_code	instrument class
instrument_type	stocks or currencies
start_date	start date
end_date	end date
interval	candle interval
is_complete	completed candles only

Custom Gold Bars

GET /api/data/custom-gold-bars

Additional parameters:

Parameter	Description
gold_ticker	gold ticker, default GLDRUB_TOM
gold_class_code	gold class, default CETS
count	number of gold units
bar_type	gold bar type

Dividends

GET /api/data/dividends

Parameters:

- figis;
- tickers;
- class_code;
- start_date;
- end_date.

Strategy API

Base path:

/api/strategies/

Health

GET /api/strategies/health

Registry

GET /api/strategies/registry

Returns groups:

- pre_selection;
- signal_model;
- allocation;
- strategy_model;
- trading_strategy_model.

Core Models

```
GET /api/strategies/models
GET /api/strategies/models/{model_name}
```

The detail endpoint returns:

- model description;
- pipeline composition;
- component groups;
- available reports.

Full Trading Strategies

```
GET /api/strategies/trading-strategies
GET /api/strategies/trading-strategies/{strategy_name}
```

CPCV

```
GET  /api/strategies/models/{model_name}/cpcv/tests
GET  /api/strategies/models/{model_name}/cpcv/tests/{test_name}
POST /api/strategies/models/{model_name}/cpcv/run
```

POST body:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "n_folds": 10,
  "n_test_folds": 6
}
```

WalkForward

```
GET  /api/strategies/models/{model_name}/walk-forward/tests
GET  /api/strategies/models/{model_name}/walk-forward/tests/{test_name}
POST /api/strategies/models/{model_name}/walk-forward/run
```

POST body:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "test_size": 0.33,
  "train_size_months": 3,
  "freq_months": 3,
  "wf_test_size": 1
}
```

Backtesting

```
GET  /api/strategies/models/{model_name}/backtest/tests
GET  /api/strategies/models/{model_name}/backtest/tests/{test_name}
POST /api/strategies/models/{model_name}/backtest/run
```

POST body:

```
{
  "test_name": "baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "trading_start_date": "2023-06-01",
  "rebalance_freq": "3ME",
  "rebalance_on": "last",
  "init_cash": 1000000,
  "fees": 0.0008,
  "slippage": 0.0,
  "freq": "1D",
  "rolling_window": 252,
  "tax_rate": 0.13
}
```

Full Trading Strategy Backtesting

```
GET  /api/strategies/trading-strategies/{strategy_name}/backtest/tests
GET  /api/strategies/trading-strategies/{strategy_name}/backtest/tests/{test_name}
POST /api/strategies/trading-strategies/{strategy_name}/backtest/run
```

Comparison

```
GET /api/strategies/comparison/latest
```

GA API

Base path:

```
/api/ga/
```

Health

```
GET /api/ga/health
```

Alphabets

```
GET /api/ga/alphabets
```

Returns gene groups and search-space size.

Runs

```
GET /api/ga/runs
GET /api/ga/runs/{run_id}
POST /api/ga/runs
```

POST body example:

```
{
  "test_name": "ga_baseline",
  "start_date": "2023-01-01",
  "end_date": "2025-12-31",
  "interval": "CANDLE_INTERVAL_DAY",
  "class_code": "TQBR",
  "test_size": 0.33,
  "num_generations": 10,
  "sol_per_pop": 8,
  "num_parents_mating": 4,
  "mutation_probability": 0.25,
  "parent_selection_type": "tournament",
  "k_tournament": 3,
  "keep_parents": 0,
  "keep_elitism": 1,
  "crossover_type": "uniform",
  "mutation_type": "random",
  "stop_criteria": "saturate_3",
  "random_seed": 42,
  "cpcv_n_folds": 4,
  "cpcv_n_test_folds": 2,
  "wf_train_size": 63,
  "wf_test_size": 21,
  "wf_purged_size": 5,
```

```
  "top_n": 3,
  "materialize_top": true
}
```

Integration Dependency

Strategy Backend and GA Backend do not call T-Invest directly. They obtain data through Data Backend. This reduces coupling and makes it possible to replace the data source without rewriting tests and GA.

Operations, Configuration, and Delivery

Delivery Package

A minimal system delivery includes:

- repository source code;
- `docker-compose.yml`;
- Dockerfiles for services;
- frontend applications;
- backend services;
- Python strategy core;
- GA engine;
- Markdown documentation in `docs`;
- PDF documentation versions in `docs/pdf`;
- interface screenshots in `docs/img`;
- tests in `tests`.

Main Launch Command

```
docker compose up --build
```

This command is the main entry point for users and clients.

PDF Documentation

Markdown files in `docs` are the primary editable documentation source. For handover as a single file, the system includes generated PDF versions:

- `docs/pdf/its_documentation_ru.pdf`;
- `docs/pdf/its_documentation_en.pdf`.

In the documentation web interface, the `PDF` button downloads the file for the currently selected interface language. After changing Markdown documentation, regenerate the PDF files with:

```
poetry run python scripts/build_docs_pdf.py
```

The script uses Markdown files and images from `docs/img`, builds Russian and English PDFs, and writes the result to `docs/pdf`.

Environment Variables

Variable	Purpose	Default
`tinvest_token`	T-Invest token	empty
`TINVEST_TOKEN`	alternative token name	empty
`TINKOFF_INVEST_API_TOKEN`	alternative token name	empty

Variable	Purpose	Default
DATA_BACKEND_STOCKS_TTL_MINUTES	instrument-reference TTL	30
DATA_BACKEND_BASE_URL	Data Backend URL for internal services	set in compose
STRATEGY_TEST_CACHE_DIR	CPCV cache	/app/its/data/strategy_tests/cpcv
STRATEGY_WF_CACHE_DIR	WalkForward cache	/app/its/data/strategy_tests/walk_forward
STRATEGY_BACKTEST_CACHE_DIR	Backtesting cache	/app/its/data/strategy_tests/backtest
GA_RUN_CACHE_DIR	GA run cache	/app/its/data/ga_runs
GA_MODELS_DIR	strategy materialization directory	/app/its/strategies/models
ITS_GATEWAY_PORT	external gateway port	8080

Data Storage

Docker Compose uses volumes:

```
t-invest-cache
strategy-test-cache
ga-cache
```

They preserve:

- loaded data across container restarts;
- expensive source responses;
- saved tests between sessions;
- GA run history.

Materialized Strategies

GA Backend mounts:

```
./its/strategies/models:/app/its/strategies/models
```

TOP strategies created by GA appear in the working copy as normal Python files.

Recommended steps after generation:

1. Review the generated file.
2. Check import in its/strategies/models/__init__.py.
3. Start Strategy Lab and verify the model is visible.
4. Run CPCV, WalkForward, and Backtesting.
5. Commit the changes if the model is accepted.

Logs

Use Docker logs:

```
docker compose logs -f
docker compose logs -f data-backend
```

```
docker compose logs -f strategy-backend
docker compose logs -f ga-backend
```

Updating the System

Recommended order:

1. Stop containers.
2. Pull or copy the new code version.
3. Check `.env`.
4. Run:

```
docker compose up --build
```

1. Check `/health` and UI.

Backup

Important artifacts:

- `its/strategies/models` - generated strategies;
- Docker volume `strategy-test-cache` - saved tests;
- Docker volume `ga-cache` - GA runs;
- Docker volume `t-invest-cache` - data cache;
- `.env` - local secrets, should not be published.

Security

The current configuration is intended for a local or closed research circuit.

Important:

- do not publish the T-Invest token;
- do not commit `.env`;
- do not expose the gateway to the public internet without authentication;
- note that backend CORS is permissive for development convenience;
- restrict server access if the system is deployed remotely.

Current Limitations

The current version:

- is not a broker terminal;
- does not place real orders;
- does not execute exchange trades;
- does not guarantee future performance;
- depends on availability and quality of the external data source.

Production execution, risk limits, order journaling, and broker integration can be added as separate services.

Client Handover

For handover, provide:

- repository or source-code archive;
- launch instructions;
- this documentation package;
- description of required tokens and access rights;
- known limitations;
- example test runs;
- list of base and generated strategies;
- demonstration CPCV, WalkForward, and Backtesting results.

Acceptance Scenario

1. Create `.env` with the token.
2. Run `docker compose up --build`.
3. Open Launchpad.
4. Open Data Hub and load quotes for `SBER`.
5. Open Strategy Lab and select a base model.
6. Run a short Backtesting test.
7. Open GA Lab and view alphabets.
8. Run a small GA search with few generations.
9. Verify that a TOP strategy appears in `its/strategies/models`.

Scientific Basis and Author Publications

ITS was developed by the author as a practical result of dissertation research. The system implements the transition from a theoretical trading-strategy model to a software platform where strategies are described through standardized components, tested, and automatically generated.

Methodological Basis

The scientific basis includes:

- component representation of trading strategies;
- architectural design using the C4 approach;
- unification of trading-model protocols;
- separation of strategy core and execution rules;
- use of genetic algorithms to search component combinations;
- use of robustness-testing methods;
- creation of a software basis for empirical validation.

Relation to Dissertation Work

Chapter 3 of the dissertation focuses on approbation of the trading-strategy formation model. It represents a transition from theoretical analysis to practical implementation.

Key ideas reflected in ITS:

- high-level software architecture diagrams were developed;
- protocols and data models for trading-strategy unification were implemented;
- a software base for implementing, testing, and improving strategies was created;
- the gap between theoretical concepts and practical application was reduced;
- a foundation for empirical validation of trading strategies was formed;
- a basis for further algorithmic-trading development was created.

Product Research Cycle

The system supports this research cycle:

1. Formulate a hypothesis.
2. Implement a component.
3. Assemble a strategy.
4. Run CPCV.
5. Run WalkForward.
6. Run Backtesting.
7. Compare models.
8. Add the component to a GA alphabet.
9. Generate new combinations.

10 Re-run empirical validation.

Author Publications

1 Aliev B. N. Analysis of Russian Stock Market Returns Through World Currency Exchange Rates // Innovations and Investments. - 2021. - No. 6. - P. 77-79. - EDN MNHCHO.

2 Aliev B. N. Evolution of the Labor Market and the Role of Artificial Intelligence on the Stock Exchange // Innovations and Investments. - 2023. - No. 10. - P. 247-252. - EDN XKAOUN.

3 Aliev B. N. Genetic Algorithms as a Tool for Formation and Evolution of Trading Strategies in the Securities Market // Finance and Credit. - 2024. - Vol. 30, No. 4(844). - P. 851-872. - EDN QZDBSH. - DOI 10.24891/fc.30.4.851.

4 Aliev B. N. Comparison of Forecast Accuracy for Classical and Alternative Price Bars in IT Companies // Scientific Research of Faculty of Economics. Electronic Journal. - 2025. - Vol. 17, No. 1(55). - P. 22-38. - DOI 10.38050/2078-3809-2025-17-1-22-38. - EDN KFGQLK.

5 Aliev B. N. Unification of Automatic Trading Systems Using the Interface Segregation Principle // Financial Analytics: Science and Experience. - 2024. - Vol. 17, No. 3(369). - P. 359-366. - DOI 10.24891/fa.17.3.359. - EDN WTIPVH.

Practical Value

The practical value of ITS is that it does not stop at methodology. It provides a working software environment where users can:

- create new models;
- run empirical validation;
- compare results;
- generate strategies;
- use software artifacts as a basis for further development.

Glossary

ITS

Intelligent Trading Strategies, a system for forming trading strategies.

Data Hub

The data subsystem. It handles sources, instruments, quotes, dividends, custom bars, and data APIs.

Strategy Lab

The model subsystem. It handles component registries, ready strategies, testing, and comparison.

GA Lab

The subsystem for generating strategies through a genetic algorithm.

Launchpad

The main program window with tiles for launching subsystems.

Pre-selection

Initial asset selection before signal and allocation stages.

Signal

A component that forms a trading signal or additional filter after pre-selection.

Allocation

A component that calculates asset weights in the portfolio.

Strategy Builder

A Python class that assembles the strategy core and returns a `Strategy` object.

Trading Strategy

A full trading strategy: portfolio-model core plus execution rules such as stop-loss and take-profit.

CPCV

Combinatorial Purged Cross-Validation, a validation method with combinatorial test folds and purging.

WalkForward

A sequential validation method where a strategy is trained on one window and tested on a following window.

Backtesting

Historical trading simulation with rebalancing, fees, slippage, and capital.

Rebalance

Periodic change of portfolio weights according to the strategy.

Equity Curve

A curve of portfolio value or cumulative return.

Drawdown

A decline from the previous portfolio-value maximum.

Sharpe Ratio

Risk-adjusted return measure calculated as mean return divided by standard deviation.

Calmar Ratio

Return relative to maximum drawdown.

CVaR

Conditional Value at Risk, an estimate of average tail loss at a confidence level.

GA

Genetic Algorithm, an algorithm that generates and selects strategies.

GeneDefinition

Description of a GA gene: component, parameters, wrapper, and runtime dependencies.

Alphabet

A gene set for one group: pre-selection, signal, or allocation.

Materialization

Creation of a Python strategy file from GA results.

FIGI

Financial Instrument Global Identifier.

TQBR

Russian equity class on Moscow Exchange, used by default.

CETS

Currency-instrument class used for `GLDRUB_TOM` and custom gold bars.

OHLCV

Candle data: open, high, low, close, volume.

Custom Gold Bar

An alternative bar built through the value of a specified amount of gold.

Software Registration Certificate

РОССИЙСКАЯ ФЕДЕРАЦИЯ

СВИДЕТЕЛЬСТВО

о государственной регистрации программы для ЭВМ

№ 2026665144

Intelligent Trading Strategies (ITS)

Правообладатель: ***Алиев Бейлак Намаз оглы (RU)***

Автор(ы): ***Алиев Бейлак Намаз оглы (RU)***

Заявка № **2026664220**
Дата поступления **02 мая 2026 г.**
Дата государственной регистрации
в Реестре программ для ЭВМ ***21 мая 2026 г.***

Руководитель Федеральной службы
по интеллектуальной собственности

ДОКУМЕНТ ПОДПИСАН ЭЛЕКТРОННОЙ ПОДПИСЬЮ
Сертификат 00a570e4f7add8d531b4b8818e75f29506
Владелец **Зубов Юрий Сергеевич**
Действителен с 04.09.2025 по 28.11.2026

Ю.С. Зубов